Technical Documentation

UNZA Hostel Maintenance Request and Tracking System

University of Zambia — Final Year Project

Technical Documentation

UNZA Hostel Maintenance Request and Tracking System

Reference documentation for developers, maintainers and examiners: how to set the system up, configure it, deploy it, operate it and extend it. For the *design rationale and diagrams* see ARCHITECTURE.md; for testing see TESTING.md; for end-user instructions see USER_MANUAL.md.

Contents

1. System summary
2. Technology and versions
3. Setup for development
4. Configuration reference
5. Database
6. Backend
7. Business rules reference
8. API reference
9. Frontend
10. PWA, offline, real-time and push
11. Background jobs and integrations
12. Security
13. Testing and CI
14. Deployment
15. Operations and troubleshooting
16. Extending the system
17. Known limitations
18. Glossary

1. System summary

A Progressive Web App for reporting and managing hostel maintenance at the University of Zambia.

Aspect	Summary
Users	Students, technicians, administrators
Architecture	Client–server; server is layered (routes → services → repositories); client is a React PWA
Rule	The browser **never** talks to Supabase; all access goes through the Express REST API
Data	Supabase PostgreSQL (with Row Level Security), Supabase Auth, Supabase Storage

Aspect	Summary
Lifecycle	`Open → Assigned → In Progress → Resolved → Closed`, enforced server-side, every transition recorded
Extras	Upvotes (incl. campus-wide visibility at 30+ votes), SLA timers + overdue alerts/reminders, live updates (SSE), Web Push, AI triage, camera-capture proof of work, dark mode, analytics with heatmap/CSV/PDF export, weekly email, admin announcements (seen by students and technicians), per-request comments, recurring-issue detection, technician performance, preventive-maintenance scheduling (clearly labelled on the technician's own view), in-app app feedback

---

2. Technology and versions

Layer	Technology	Version
Runtime	Node.js	22 (used in development and CI)
Server framework	Express	5.x
Database client	`@supabase/supabase-js`	2.x
Web push	`web-push`	3.x
Email	`nodemailer`	10.x
AI (optional)	`@anthropic-ai/sdk`	0.127
Server tests	Vitest 4, Supertest 7	
Front end	React	19.x
Routing	`react-router-dom`	7.x
Charts	Recharts	3.x (only chart library)
Build tool	Vite	8.x
Lint	oxlint	
Language	JavaScript (ES modules) end to end	
Styling	Plain CSS, design tokens as CSS variables	

---

3. Setup for development

Prerequisites

Node.js 22+, npm, a Supabase project (free tier is enough), Git.

Steps

```
# 1. Get the code
git clone <repository-url>
cd FINAL_YEAR_PROJECT_SAMSON

# 2. Server
cd server
npm install
cp .env.example .env          # then fill in the values (section 4)

# 3. Client
cd ../client
npm install
cp .env.example .env          # VITE_API_BASE_URL=http://localhost:5000/api
```

Create the database

In the Supabase dashboard → **SQL Editor**, run the files in `server/sql/` **in numeric order** (`001` … `016`); see section 5.4. Then create the first administrator account (section 15, "First administrator").

Run

```
# terminal 1
cd server && npm run dev      # API on http://localhost:5000

# terminal 2
cd client && npm run dev      # app on http://localhost:5173
```

Health check: `GET http://localhost:5000/api/health` → `{"status":"ok", ...}`.

Useful commands

Where	Command	Purpose
`server`	`npm test` / `npm run test:unit` / `npm run test:integration`	Tests
`server`	`npm start`	Production start (`node server.js`)
`client`	`npm run build`	Production build into `client/dist`
`client`	`npm run preview`	Serve the production build locally
`client`	`npm run lint`	Lint

---

4. Configuration reference

Environment files are **never committed** (`.env` is git-ignored); `.env.example` files document every variable.

Server (`server/.env`)

Variable	Required	Purpose
`PORT`	no (5000)	HTTP port. Render sets this automatically
`NODE_ENV`	no	`development` / `production` (controls request logging)
`SUPABASE_URL`	**yes**	Project URL from Supabase → Project Settings → API
`SUPABASE_SERVICE_ROLE_KEY`	**yes**	Service-role key. **Secret — server only, never the client**
`VAPID_PUBLIC_KEY` / `VAPID_PRIVATE_KEY`	for push	Generate with `npx web-push generate-vapid-keys` . Empty = push disabled
`VAPID_SUBJECT`	for push	Contact, e.g. `mailto:you@example.com`
`ANTHROPIC_API_KEY`	optional	Enables AI triage suggestions. Empty = keyword rules only
`ANTHROPIC_MODEL`	optional	Defaults to `claude-opus-5`
`SMTP_HOST` , `SMTP_PORT` , `SMTP_USER` , `SMTP_PASS`	for email	Weekly summary. Gmail: `smtp.gmail.com` , `465` , your address, an *App password*
`MAIL_FROM`	optional	Sender display, e.g. `UNZA Hostel Maintenance <you@gmail.com>`
`APP_URL`	for email	Public URL of the web app, used for links in emails
`REQUIRE_RESOLUTION_PROOF`	optional	`true` makes the resolution photo mandatory for technicians (default: optional)

Missing optional variables **disable the feature cleanly**; the rest of the app keeps working.

Client (`client/.env`)

Variable	Purpose
`VITE_API_BASE_URL`	Base URL of the API, e.g. `https://<service>.onrender.com/api` . Baked in at **build time** — rebuild after changing it

The client holds **no Supabase key** and no secrets.

---

5. Database

5.1 Enumerations

Type	Values
`user_role`	`student`, `technician`, `administrator`
`request_status`	`open`, `assigned`, `in_progress`, `resolved`, `closed`
`request_priority`	`low`, `medium`, `high`, `critical`
`technician_availability`	`available`, `on_job`, `off_duty`

5.2 Data dictionary

users — one row per account (mirrors Supabase Auth)

Column	Type	Notes
`user_id`	uuid PK	= `auth.users.id`
`name`	text	
`email`	text unique	Supabase Auth identifier
`id_number`	text unique	Student/staff ID; the default login credential
`role`	user_role	default `student`
`availability_status`	technician_availability	default `available`; meaningful for technicians
`avatar_url`	text	profile photo
`hostel`	text null	student's hostel (migration `013`); set in Settings or learned from their first request; targets announcements
`created_at`	timestamptz	

categories — `category_id` PK, `name` unique, `description`

maintenance_requests

Column	Type	Notes
`request_id`	serial PK	
`student_id`	uuid FK → users	who reported
`category_id`	int FK → categories	
`description`	text	
`photo_url`	text null	issue photo
`status`	request_status	default `open`

Column	Type	Notes
`priority`	request_priority	default `medium`
`hostel` , `room`	text	free text; hostel list is a client-side dropdown
`overdue_notified_at`	timestamptz null	time of the most recent overdue alert (drives reminder spacing)
`created_at`	timestamptz	starts the SLA clock

technician_assignments — `assignment_id` PK, `request_id` FK, `technician_id` FK → users, `assigned_at`

status_history — `history_id` PK, `request_id` FK, `previous_status` (null for the first row), `new_status` , `changed_at` . **One row per transition**; feeds the timeline and analytics.

ratings — `rating_id` PK, `request_id` FK **unique**, `student_id` FK, `rating` 1–5, `comment` , `created_at`

request_upvotes — composite PK (`request_id` , `user_id`); a duplicate vote is impossible at the DB level

resolution_proofs — `request_id` PK/FK, `photo_url` , `note` , `submitted_by` FK, `created_at` (one per request)

push_subscriptions — `subscription_id` PK, `user_id` FK, `endpoint` **unique**, `p256dh` , `auth` , `created_at` (one row per device)

job_runs — `job_name` PK, `last_run_at` (prevents duplicate scheduled jobs after a restart)

announcements — `announcement_id` PK, `title` , `message` , `hostel` (null = all hostels), `created_by` FK, `created_at` , `expires_at` (null = never)

request_comments — `comment_id` PK, `request_id` FK, `author_id` FK, `body` , `created_at` (one row per message; ordered oldest-first for the conversation view)

maintenance_schedules — `schedule_id` PK, `title` , `description` , `category_id` FK, `hostel` , `room` , `priority` , `frequency_days` (1–730), `next_due_at` , `active` , `created_by` FK, `created_at` , `last_generated_at`

maintenance_schedule_runs — `run_id` PK, `schedule_id` FK, `request_id` FK, `created_at` (which request each schedule produced, so a new task is never created while the previous one is still open)

app_feedback — `feedback_id` PK, `user_id` FK, `rating` 1–5, `category` (`bug` | `suggestion` | `general` , default `general`), `message` text null, `created_at` . A review of the **app itself** (not of one technician's work — that is `ratings` above); exists so a student, who has no direct line to the developer the way an administrator does, still has a channel to be heard.

5.3 Security in the database

- **Row Level Security is enabled on every table**, as a second line of defence behind the server: students see only their own rows, technicians only their assigned work, administrators broad access. Helper: `public.current_user_role()` .
- Foreign keys enforce referential integrity everywhere.
- The server connects with the **service role**, which bypasses RLS — which is why the server performs its own role and ownership checks on every request.

5.4 Migrations

Run **in order** in the Supabase SQL Editor. Each file is written to be run once.

File	Purpose
`001_init_schema.sql`	Core tables, enums, RLS policies
`002_add_id_number.sql`	Adds `users.id_number` (unique) for ID-based login
`003_grant_service_role_privileges.sql`	Table grants for the server's service role
`004_seed_categories.sql`	Default categories (Plumbing, Electrical, Furniture)
`005_create_request_photos_bucket.sql`	Storage bucket `request-photos` (public read, JPEG/PNG/WebP only, 5 MB limit)
`006_backfill_submitted_status_history.sql`	Backfills the initial `open` history row for older requests (idempotent)
`007_add_priority_location_ratings_availability.sql`	Priority, hostel/room, ratings, technician availability
`008_add_avatar.sql`	`users.avatar_url` for profile photos. **The `avatars` bucket itself is created in the Supabase Storage dashboard** (public, 3 MB, JPEG/PNG/WebP) — see the note in the file
`009_request_upvotes.sql`	"Me too" upvotes
`010_push_subscriptions.sql`	Web Push subscriptions
`011_overdue_alerts_and_job_runs.sql`	`overdue_notified_at`, `job_runs`
`012_resolution_proofs.sql`	Proof-of-work photos and notes
`013_announcements_and_user_hostel.sql`	`announcements` table, `users.hostel`. **Safe to re-run**
`014_request_comments.sql`	`request_comments`. **Safe to re-run**
`015_preventive_maintenance.sql`	`maintenance_schedules`, `maintenance_schedule_runs`. **Safe to re-run**
`016_app_feedback.sql`	`app_feedback`. **Safe to re-run**

> Categories are managed in the `categories` table. Add e.g. *Carpentry* or *Internet/IT* by inserting rows; the UI picks them up. Migrations `013`–`016` use `create table if not exists` / `drop policy if exists` so running them again is harmless — useful if you are not sure whether one already ran.

5.5 Storage

Bucket	Contents	Paths
`request-photos`	Issue photos and proof-of-work photos	`<studentId>/<uuid>.<ext>`, `resolutions/<requestId>/<uuid>.<ext>`

Bucket	Contents	Paths
`avatars`	Profile pictures	`<userId>/avatar.<ext>`

Uploads happen **server-side** from base64 in the JSON body: JPEG/PNG/WebP only, ≤ 5 MB (3 MB for avatars).

---

6. Backend

6.1 Layers and module map

```
server/
  server.js                 starts the listener + scheduler
  src/app.js                Express app (importable by tests)
  src/routes/               PRESENTATION: validate, authenticate, authorise, call ONE service
  src/services/             BUSINESS LOGIC: rules; no req/res; unit-testable
  src/repositories/         DATA ACCESS: the only files that import Supabase
  src/middleware/           authenticate, requireRole, errorHandler
  src/jobs/scheduler.js     background timers (overdue, weekly email, preventive maintenance)
  src/config/               env, supabaseClient, webPush, mailer, anthropic
  src/utils/                sla.js (pure SLA + reminder rule), preventive.js, labels.js,
httpError.js
```

Layers only call **downward**. A route never touches the database; a repository never contains a business rule.

Service	Responsibility
`auth.service`	Register students, ID→email login, JWT verification (30 s profile cache)
`requests.service`	Lifecycle rules, submission, assignment, status changes, proof of work, detail/list mapping
`upvotes.service`	Community feed, add/remove upvotes
`triage.service`	Category/priority suggestion (Claude or rules) + safety floor + rate limit
`users.service`	Profiles, technician ranking, availability
`reports.service`	Analytics maths (resolution time, workload, trends, heatmap inputs)
`overdue.service`	Overdue detection, first alerts and repeat reminders
`digest.service`	Weekly statistics, email template, schedule
`push.service`	Subscribe/unsubscribe, deliver, clean up dead subscriptions
`events.service`	Live-update hub (Server-Sent Events)

Service	Responsibility
`announcements.service`	Create/list/delete announcements, audience resolution, a student's hostel
`comments.service`	Per-request conversation (student/assigned technician/admins only), notifications
`recurrence.service`	Detects faults repeating at the same room or across a hostel
`performance.service`	Per-technician resolution/on-time/rating maths over a time window
`schedules.service`	Preventive-maintenance schedules; creates requests when they fall due
`feedback.service`	App-itself reviews (rating + category + comment), rate limiting, admin listing with the average rating
`ratings.service`, `categories.service`, `activity.service`	Ratings, category list, recent activity

6.2 Request pipeline

`cors` → `compression` (skipped for `/api/events`) → `express.json` (8 MB, for base64 photos) → request logging → route → `authenticate` (JWT → profile) → `requireRole(...)` → validation → **service** → repositories → JSON.

6.3 Response and error format

Success: `{ "status": "ok", ...payload }`. Errors: `{ "status": "error", "message": "..." }` with an HTTP status:

Status	Meaning
400	Validation failed, or a business rule rejected the action (e.g. invalid transition)
401	Missing/invalid/expired token
403	Authenticated but not allowed (wrong role, or not your resource)
404	Not found
429	Rate limit (AI suggestions)
503	Optional feature not configured (e.g. email)
500	Unexpected server error (logged)

---

7. Business rules reference

7.1 Ticket lifecycle

`open → assigned → in_progress → resolved → closed`. One step at a time; no skipping, no going back, no changes after `closed`.

- `assigned` is reachable **only** through the assign action (creates the assignment row in the same operation).
- The status endpoint accepts `in_progress`, `resolved`, `closed`.
- A **technician** may advance only requests assigned to them; an **administrator** may advance any.
- Every transition writes a `status_history` row.

7.2 SLA (target resolution time)

Priority	Target
critical	4 h
high	24 h
medium	72 h
low	168 h (1 week)

State: `on_track`; `at_risk` when < 25 % of the window remains; `overdue` when past due; `done` once resolved/closed. Computed by one pure function (`utils/sla.js`) and attached as `sla` to every request returned by the API.

7.3 Overdue alerts

There are **two intervals**:

Interval	Value	Meaning
Check interval	every **1 hour**	How often the server looks for overdue requests
Reminder interval	**4 hours** for `critical` and `high`; **24 hours** (daily) for `medium` and `low`	Minimum gap between alerts about the *same* request while it stays overdue

A request is alerted when it first becomes overdue, then reminded at the reminder interval until it is resolved or closed. `overdue_notified_at` stores the time of the **most recent** alert; it is written **before** sending, so a slow or failed send cannot cause a repeat. A 5-minute tolerance stops timer drift from delaying a 4-hour reminder to 5 hours. Reminders are worded "still overdue". Admins and the assigned technician are notified (push + live refresh); more than 3 at once collapse into one summary notification. Values live in `OVERDUE_REMINDER_HOURS` (`server/src/utils/sla.js`).

7.4 Technician recommendation

Order by availability (`available` < `on_job` < `off_duty`), then fewest active tasks (assigned/in-progress), then name. The first entry is `recommended` unless that technician is off duty.

7.5 Upvotes ("Me too")

Students only; not on own requests; only on `open/assigned/in_progress` ; idempotent; the community feed shows other students' active requests for the student's hostel (defaults to the hostel set in Settings, or of their latest request), ranked by votes then recency, capped at 12, **with no names or ids**. A request with `CAMPUS_WIDE_UPVOTE_THRESHOLD` (30) or more upvotes is shown to students in **every** hostel, not just its own, flagged `campusWide: true` — a fault that widespread is unlikely to be confined to one building.

7.6 AI / rules triage

- With `ANTHROPIC_API_KEY` : Claude returns schema-constrained JSON `{categoryId, priority, reason}` ; on any failure the rules engine answers.
- **Safety floor** (both engines): hazard words (fire, smoke, sparking, electrocution, exposed wires, flooding, burst, gas leak, sewage, collapse) → at least `critical` ; loss of service (no water/power, locked out, broken lock/window/door) → at least `high` .
- Description is treated as untrusted; max 1000 characters; 10 suggestions per user per minute.
- Advisory only: nothing is saved.

7.7 Proof of work

When resolving, a technician may attach a photo and note. The photo is validated and stored **before** the status changes. Optional by default; `REQUIRE_RESOLUTION_PROOF=true` makes it mandatory for technicians (administrators always exempt).

7.8 Who is notified of what

Event	Live update	Push
New request	admins, the student	admins
Assigned	student, technician, admins	student, technician
Status change	student, assigned technician, admins	student
Becomes overdue	admins, technician	admins, technician
New comment	the other people in the conversation (never the author)	same
New announcement	students targeted by its hostel (or all)	same

Push delivery never blocks or fails the action that triggered it.

7.9 Admin announcements

Title 3–80 chars, message 5–500 chars, optional expiry (must be in the future). `hostel: null` targets every student; a named hostel targets students who **chose** it in Settings **plus** students whose **latest request** was

there (deduplicated) — so a student never has to set their hostel manually to receive notices, though doing so is more reliable. Sending is a side effect (never blocks creating the announcement); a student sees an announcement if it is not expired and (`hostel === null` or `hostel === their hostel`). A **technician** sees every active announcement regardless of hostel (`selectForTechnician`) — unlike a student, a technician is not tied to one hostel and can be assigned a task anywhere, so a hostel-scoped notice ("water off in Block C") is still relevant context for them.

7.10 Comments on a request

Only the reporting student, the currently assigned technician, and administrators may read or post (403 otherwise) — exactly the same audience as reading the request itself. Body 1–1000 chars after trimming; no comments once a request is `closed` ; 15 posts/minute per user. The author is always the authenticated user, never a request-body field. Posting notifies the *other* people in the conversation, never the author.

7.11 Recurring issue detection

Pure function over recent requests (`recurrence.service.detectRecurringIssues`), excluding preventive-maintenance-generated requests (7.13). Two levels:

Scope	Threshold	Window
Same room, same category	3+ reports	60 days
Same hostel, same category (any room)	6+ reports	30 days

Room text is normalised (case/whitespace) so "Block C, Room 24" and "block c, room 24" count as the same room. `severity` is `high` at double the threshold, otherwise `watch` . Results are sorted most-frequent first. Each result links to **Plan a preventive check**, prefilling a new schedule (7.13) with the matching category/hostel/room.

7.12 Technician performance

Pure function (`performance.service.computeTechnicianPerformance`) over a chosen window (7/30/90/365 days, or 0 = all time — `ALLOWED_WINDOWS`). Per technician: `resolved` (finished in the window), `avgResolutionHours` (measured from **assignment**, not submission — so time an administrator took to assign is not counted against the technician), `onTimeRate` (% resolved within the request's SLA target, measured end-to-end from submission), `active` jobs on hand, `overdueNow` , and `avgRating` from students. A reassigned request counts only for the most recent assignee. No work in the window gives `null` averages, never `0` or `NaN` . Administrators see every technician (`GET /reports/technician-performance`); a technician sees only their own numbers plus their rank (`GET /reports/my-performance`).

7.13 Preventive maintenance

An administrator-defined recurring task (`maintenance_schedules` : category, hostel, optional room, priority, frequency in days, next due date — one of `7, 14, 21, 30, 90, 180, 365` days, `FREQUENCY_DAYS`). An hourly job (`schedules.service.runDueSchedules`) finds schedules whose `next_due_at` has passed and, for each: **advances** `next_due_at` **first** (so a failure cannot cause the same schedule to fire again on the next hourly check), then — **only if the previous task it generated is finished** (not `open` / `assigned` / `in_progress`) — creates a normal maintenance request via the same `requests.service.createRequest` path used for student

submissions (so it gets a `status_history` row, SLA, admin notification, everything). The request is reported by the administrator who owns the schedule and its description is prefixed `"Preventive maintenance: "` (`utils/preventive.js`), which excludes it from the student "Already reported" feed (7.5), from recurring-issue detection (7.11), and from learning a hostel for the reporting admin. A schedule resumed after being paused jumps its due date to the next future occurrence rather than firing immediately for a stale date. An administrator can also **run a schedule immediately** (`POST /schedules/:id/run-now`), subject to the same "previous task finished" rule.

On the assigned **technician's** side, a task generated this way is visually distinguished from a student-reported fault rather than left to blend in: it appears in its own "My Maintenance Jobs" panel on the dashboard (not mixed into "My Active Tasks"), carries a client-side "Recurring Maintenance" badge wherever it appears (dashboard, My Tasks, task detail — `components/PreventiveBadge.jsx`, detected the same way as the server, by the description prefix), has the prefix itself stripped from the displayed description now that the badge says it (`displayDescription()` in `utils/requestFormat.js` — display-only, the stored description is untouched), and its "Reported By" section reads "Maintenance Schedule" with a plain explanation instead of naming the administrator as if they had personally filed it.

7.14 App feedback

A review of the **app itself** — a star rating (1–5, required) plus a category (`bug` | `suggestion` | `general`, defaults to `general` if unrecognised) and an optional comment (≤ 1000 chars) — separate from `ratings` (7.7's "Rate This Repair", which rates one technician's work on one resolved request). Any authenticated role may submit one; rate-limited to 5 per hour per user (same in-memory pattern as comments, 7.10). Every submission sends a non-blocking push notification to administrators. A user can read back their own submission history (`GET /feedback/mine`); only administrators can list every submission (`GET /feedback`), which also returns the average rating so far. This exists because an administrator already has a direct line to the developer as the project's stakeholder, but a student typically does not — this is that channel.

---

8. API reference

Base path `/api`. Protected endpoints require `Authorization: Bearer <access_token>`. Bodies are JSON. `:id` is the numeric `request_id`.

8.1 Authentication

Method & path	Auth	Body	Response
`POST /auth/register`	public	`name`, `email`, `id_number`, `password` (≥ 8 chars)	`201 { user }` (student only)
`POST /auth/login`	public	`password` and **either** `id_number` **or** `email`	`200 { session: {access_token, refresh_token, ...}, user }`
`GET /auth/me`	any	—	`{ user }`

8.2 Requests

Method & path	Role	Body / notes	Response
`POST /requests`	student	`categoryId`, `description`, `hostel` (required); `room`, `priority` (`low…critical`), `photo` (data URL)	`201 { request }`
`GET /requests/mine`	student	own requests	`{ requests }`
`GET /requests/community`	student	optional `?hostel=`	`{ hostel, requests }` (no identities)
`POST /requests/triage`	student	`description` (≥ 10 chars)	`{ suggestion: {categoryId, priority, reason, source} }`
`POST /requests/:id/upvote` · `DELETE /requests/:id/upvote`	student	—	`{ requestId, hasUpvoted, upvoteCount }`
`POST /requests/:id/rating`	student	`rating` 1–5, `comment?`	`201 { rating }` (own, resolved/closed, once)
`GET /requests`	administrator	all requests (each has `upvoteCount`)	`{ requests }`
`POST /requests/:id/assign`	administrator	`technicianId` (uuid)	`{ requestId, status:"assigned", technicianId }`
`PATCH /requests/:id/priority`	administrator	`priority`	`{ requestId, priority }`
`GET /requests/assigned-to-me`	technician	(each has `upvoteCount`)	`{ requests }`
`GET /requests/:id`	scoped*		`{ request }` incl. `history`, `rating`, `upvoteCount`, `resolution`
`PATCH /requests/:id/status`	technician (assigned) / administrator	`status`; optional `resolutionPhoto` (data URL), `resolutionNote`	`{ requestId, status }`
`GET /requests/:id/comments` · `POST /requests/:id/comments`	scoped*	`POST` body: `{ body }` (1–1000 chars)	`{ comments }` / `201 { comment }`

*owner student, assigned technician, or administrator.

Request object (returned by list/detail): `requestId, description, photoUrl, status, hostel, room, priority, createdAt, sla{targetHours,dueAt,state,hoursRemaining}, category{id,name},`

`student{id,name,idNumber}` (staff views), `technician{id,name,availabilityStatus}`.

8.3 Users, categories, activity

Method & path	Role	Notes
`GET /users`	administrator	all accounts
`POST /users`	administrator	`name`, `id_number`, `email`, `password` (≥ 8), `role` (`technician` / `administrator`) → `201 { user }`
`GET /users/technicians`	administrator	ranked; each has `activeTasks`, `recommended`
`GET /users/me` · `PATCH /users/me`	any	`PATCH` accepts `name`, `photo` (data URL)
`PATCH /users/me/availability`	technician	`availabilityStatus`
`GET /categories`	any	`{ categories: [{id, name, description}] }`
`GET /activity/recent`	any	role-scoped recent status changes

8.4 Reports

Method & path	Role	Response
`GET /reports/analytics`	administrator	`averageResolutionHours, averageRating, thisMonth{totalRequests,resolved}, requestsByCategory, requestsByHostel, workloadByTechnician, resolutionTrend`
`POST /reports/digest/send`	administrator	emails the weekly summary to the caller → `{ sentTo }` (`503` if email not configured)
`GET /reports/recurring`	administrator	`{ issues }` — faults repeating at a room or hostel level (7.11)
`GET /reports/technician-performance?days=`	administrator	`?days` one of `7,30,90,365,0` (default 30) → `{ days, technicians }` (7.12)
`GET /reports/my-performance?days=`	technician	own numbers + rank → `{ days, performance, rank, outOf }`

8.5 Announcements

Method & path	Role	Body / notes	Response
`POST /announcements`	administrator	`title`, `message`, `hostel` (omit/null = all), `expiresAt?` (ISO, future)	`201 { announcement, recipients }`
`GET /announcements`	any	students: only what applies to them; administrators: everything, each flagged `active`	`{ announcements }`

Method & path	Role	Body / notes	Response
`DELETE /announcements/:id`	administrator	—	`{ id, deleted: true }`
`GET /announcements/hostel/me` · `PUT /announcements/hostel/me`	student	`PUT` body `{ hostel }`	`{ hostel }`

8.6 Preventive maintenance schedules

All administrator-only (`server/src/routes/schedules.routes.js` applies `requireRole('administrator')` to the whole router).

Method & path	Body / notes	Response
`GET /schedules`		`{ schedules, frequencies: [7,14,21,30,90,180,365] }`
`POST /schedules`	`title` , `categoryId` , `hostel` , `frequencyDays` ; optional `description` , `room` , `priority` , `firstDueAt`	`201 { schedule }`
`PATCH /schedules/:id`	`{ active }` — pause/resume	`{ id, active }`
`DELETE /schedules/:id`	(existing generated tasks are kept)	`{ id, deleted: true }`
`POST /schedules/:id/run-now`	creates the task immediately; `400` if the previous one is still open	`201 { scheduleId, requestId }`

8.7 Feedback

Method & path	Role	Body / notes	Response
`POST /feedback`	any	`rating` (1–5), `category?` (`bug` / `suggestion` / `general`), `message?` (≤ 1000 chars)	`201 { feedback }`
`GET /feedback/mine`	any	the caller's own submissions	`{ items }`
`GET /feedback`	administrator	every submission	`{ items, averageRating, count }`

8.8 Live updates and push

Method & path	Notes
`GET /events`	**Server-Sent Events** stream (`text/event-stream`), authenticated. Events: `ready` , `request` with `{type:"request_changed", requestId, message, actorId}` . Heartbeat every 25 s

Method & path	Notes
`GET /push/public-key`	public; `{ publicKey }` (`null` = push disabled)
`POST /push/subscribe`	`{ subscription }` = the browser's `PushSubscription` JSON
`POST /push/unsubscribe`	`{ endpoint }`
`GET /health`	public liveness probe

---

9. Frontend

9.1 Routes

Path	Page	Roles
`/login` , `/register`	Login, Register	public
`/`	Home (role-specific dashboard)	all
`/requests` , `/requests/new` , `/requests/:id`	My Requests, Report Issue, Request Detail	student
`/tasks` , `/tasks/:id`	My Tasks, Task Detail	technician
`/manage-requests` , `/manage-requests/:id`	Manage Requests (incl. Overdue tab), Admin Request Detail	administrator
`/user-management`	User Management	administrator
`/announcements`	Announcements	administrator
`/preventive-maintenance`	Preventive Maintenance	administrator
`/reports`	Reports & Analytics (incl. Technician Performance, Needs a Permanent Fix)	administrator
`/profile`	Account Settings	all
`/feedback`	Feedback (rate the app + own submission history)	student, technician
`/app-feedback`	App Feedback (every submission + average rating)	administrator

`ProtectedRoute` redirects and hides pages by role — a **convenience only**; the server enforces access.

9.2 Folder responsibilities

```
client/src/
  pages/        one folder per screen (Page.jsx + Page.css)
  components/   shared UI: Sidebar, TopNav, StatCard, SlaBadge, AffectedBadge,
                PreventiveBadge (marks a technician's recurring-maintenance jobs), charts,
                UpvoteButton, CommunityIssues, SimilarIssues, TriageSuggestion,
                ResolutionProof(Form), PhotoInput (camera capture / file picker),
                HostelHeatmap, RecurringIssues, TechnicianPerformance, MyPerformance,
                AnnouncementBanner, HostelSetting, CommentsThread,
                NotificationSettings, ThemeToggle
  services/     api.js (fetch wrapper), apiCache.js, requests/users/reports/
                announcements/schedules/feedback/... service modules, draftQueue/draftSync
                (offline), pushService, liveEvents, theme
  hooks/        useAuth (session), useLive (live updates), useDraftSync
  styles/       variables.css (design tokens), theme.css (dark theme)
  utils/        requestFormat (labels/icons), csv, image (photo compression)
```

9.3 State and data flow

- **Auth**: `AuthProvider` holds the user; tokens live in `localStorage` (if "Remember this device") or `sessionStorage`.
- **API layer**: all calls go through `services/api.js` (adds the bearer token, normalises errors, flags network failures).
- **Stale-while-revalidate cache** (`apiCache.js`): each `getX()` has a `.peek()` ; pages seed their state from it, then refetch. The cache is cleared on any write and on login/logout.
- **Live updates**: `LiveProvider` keeps one SSE connection; each event clears the cache and bumps `liveVersion` , which pages include in their fetch effects so they refetch themselves.
- **Code splitting**: every page is a lazy chunk; remaining pages are prefetched when the browser is idle; the chart library loads only for admins.

9.4 Styling conventions

- Plain CSS, one file per component/page; **class names are prefixed with the component name** (`.request-card-title`) to avoid global clashes.
- All colours, fonts and spacing are CSS custom properties in `variables.css` ; dark mode overrides them in `theme.css` (`html[data-theme="dark"]`).
- **Mobile-first**: default styles target 375 px (must not break at 320 px); `min-width` media queries at 768 px (tablet) and 1280 px (desktop). Login/Register switch to two panels at 768 px.

---

10. PWA, offline, real-time and push

Piece	File	Behaviour
Manifest	`client/public/manifest.json`	Name, PNG + maskable icons, standalone display, shortcuts (*Report an issue*, *My requests*)
Service worker	`client/public/service-worker.js`	`/assets/*` **cache-first**; page loads **network-first** with a 2.5 s timeout then cached shell; other GETs network-first; handles `push` and `notificationclick`
Offline drafts	`services/draftQueue.js`, `draftSync.js`, `hooks/useDraftSync.js`	New requests written offline are stored in IndexedDB and submitted when the browser fires `online` (implemented in-page because Background Sync is unavailable on iOS/Firefox)
Session resilience	`services/api.js`, `hooks/useAuth.jsx`	`navigator.onLine` only reflects the network adapter, not real internet access, so a failed fetch that *might* just be a bad connection now says "check your internet connection" rather than confidently blaming the server. A failed session check on app load no longer clears a valid session over a network blip — only a genuine auth rejection does; a network failure retries automatically on the browser's `online` event, and `Login.jsx` redirects away once that retry restores the session
Live updates	`services/liveEvents.js`, `hooks/useLive.jsx`, server `events.*`	SSE via `fetch` (EventSource can't send the bearer token); exponential-backoff reconnect; catches up on reconnect / tab focus
Push	`services/pushService.js`, server `push.*`	VAPID Web Push; per-device subscription; logout removes the subscription; re-enabled silently on next login if the user had opted in
Theme	`services/theme.js`, `index.html`	`system` / `light` / `dark`; applied before first paint

After a deployment, users may need one reload to pick up a new service worker.

---

11. Background jobs and integrations

Jobs run as plain timers inside the Express process (`src/jobs/scheduler.js`); each catches its own errors.

Job	Schedule	Detail
Overdue monitor	first run 30 s after start, then **every hour** (check interval); reminders every 4 h / 24 h by priority	See 7.3
Weekly summary email	checked hourly; sends Monday ≥ 06:00 UTC (08:00 Lusaka)	Catches up if a week was missed; run recorded in `job_runs` **before** sending so a failure/restart cannot duplicate

Job	Schedule	Detail
Preventive maintenance	first run 30 s after start, then **every hour**	Creates a request for each schedule whose `next_due_at` has passed, unless the previous one it generated is still open. See 7.13

Integrations degrade gracefully:

Integration	If not configured
Web Push	Subscribing returns 503; nothing else affected
Email	Weekly job skipped; "Email me the summary" returns 503 with a clear message
Claude (AI)	Keyword-rules triage is used

---

12. Security

Area	Measure
Authentication	Supabase Auth JWT verified server-side on every protected route; ID→email resolution happens on the server
Authorisation	`requireRole` on routes **and** ownership checks in services; RLS as a second layer
Secrets	`.env` git-ignored; service-role key only on the server; the client has no Supabase key
Input	Validated on the server regardless of client validation; uploads restricted to JPEG/PNG/WebP, ≤ 5 MB
Privacy	Community feed exposes no identities; students see only their own requests
Output	Email templates HTML-escape user text; React escapes output by default
AI	Descriptions treated as untrusted data; safety floor cannot be overridden by the model; per-user rate limit
Sessions	Push subscription removed on logout; API cache cleared on login/logout (no cross-user leakage on shared devices)
Transport	HTTPS in deployment (Netlify and Render provide it)
Hardening to consider	Restrict CORS to the deployed front-end origin (currently open); add request-rate limiting on login; enable Supabase email confirmation

---

13. Testing and CI

275 automated tests (199 unit + 76 integration). Run `npm test` in `server/`. GitHub Actions (`.github/workflows/ci.yml`) runs the server tests and the client lint + build on every push and pull request, with no secrets. Full detail in TESTING.md.

---

14. Deployment

Recommended: **Render** (API) + **Netlify** (front end) + **Supabase** (data).

14.1 Order of operations

1. Run all SQL migrations in Supabase (section 5.4).
2. Deploy the API (Render) and note its URL.
3. Deploy the front end (Netlify) with `VITE_API_BASE_URL` pointing at the API.
4. Smoke-test (14.4).

14.2 API on Render

1. **New** → **Web Service** → connect the GitHub repository.
2. Settings: **Root Directory** `server` · **Build Command** `npm install` · **Start Command** `node server.js`.
3. **Environment**: `NODE_ENV=production`, `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, plus any optional variables from section 4 (VAPID keys, SMTP, `APP_URL`, `ANTHROPIC_API_KEY`). Do **not** set `PORT`.
4. Deploy, then open `https://<service>.onrender.com/api/health`.

14.3 Front end on Netlify

1. **Add new site** → **Import from Git**.
2. Settings: **Base directory** `client` · **Build command** `npm run build` · **Publish directory** `client/dist`.
3. **Environment variable**: `VITE_API_BASE_URL=https://<service>.onrender.com/api`, then **redeploy** (Vite bakes it in at build time).
4. `client/public/_redirects` (`/* /index.html 200`) makes deep links and page refreshes work.
5. Set the same public URL as `APP_URL` on the server so emails link correctly.

14.4 Smoke test

Health endpoint OK → sign in as each role → student submits a request → admin sees it (live) and assigns → technician resolves → student sees the result. On a phone: install to Home Screen, enable notifications.

14.5 Keeping the free tier responsive

Render's free tier sleeps after ~15 minutes idle, making the first request slow. Point an uptime monitor (e.g. UptimeRobot) at `/api/health` every 5–10 minutes. Background jobs also only run while the server is awake.

14.6 CI/CD

Netlify and Render redeploy automatically on each push to the connected branch. The GitHub Actions pipeline runs alongside; optionally require it to pass before merging.

---

15. Operations and troubleshooting

First administrator

Administrators are created by other administrators (User Management). For the **first** one: create a user in Supabase → Authentication, then insert the matching row in `public.users` with `role = 'administrator'` and an `id_number` (or register a student and change the role in the table).

Symptom → cause → fix

Symptom	Likely cause	Fix
Login says "Could not reach the server"	API not running / wrong `VITE_API_BASE_URL`	Start the server; check the URL; rebuild the client after changing env
A new feature's page errors / route returns 404	Server not restarted after an update	Restart the API
"more than one relationship was found for 'maintenance_requests' and 'users'"	Old server code running after migration `009`	Restart the API with the current code (queries use the `student_id` hint)
"relation ... does not exist" / "column ... does not exist"	A migration hasn't been run	Run the missing file from `server/sql` in order
"permission denied for table ..."	Service-role grants missing	Run `003_grant_service_role_privileges.sql`
Resolving fails only when a photo is attached	`012` not run, or `request-photos` bucket missing	Run `012` / `005`
Overdue monitor logs "(has sql/011 been run?)"	Migration `011` missing	Run it
Preventive maintenance monitor logs "(has sql/015 been run?)"	Migration `015` missing	Run it
Posting an announcement / comment fails, or a student's hostel won't save	Migration `013` / `014` missing	Run it (both are safe to re-run)
Sending app feedback fails	Migration `016` missing	Run it (safe to re-run)
"Email is not configured" (503)	SMTP variables missing	Set `SMTP_*`, `APP_URL`; restart
Notifications toggle greyed out / "blocked"	Browser permission denied, or unsupported browser	Allow notifications for the site; on iPhone add to Home Screen first
"Push notifications are not enabled on the server yet"	VAPID keys missing	Set `VAPID_*` and restart
AI suggestions say "Suggestion" not "AI suggestion"	No `ANTHROPIC_API_KEY` (rules engine in use)	Add the key (optional)
UI looks old after a deployment	Cached service worker	Reload once (or hard-refresh); check the new version is deployed

Symptom	Likely cause	Fix
First request after idle is very slow	Render free tier cold start	Add a health-check ping (14.5)
Port already in use on start	Old server still running	Stop the old process, or set another `PORT`

Logs

The server logs to stdout (Render → **Logs**). Errors pass through the central error handler and are logged with their stack.

Backups

Use Supabase's dashboard backups. Uploaded photos live in Supabase Storage.

---

16. Extending the system

Add a field to requests (e.g. "urgent flag")

1. New migration in `server/sql/` (`017_...sql`), run it in Supabase.
2. Repository: add the column to the relevant `SELECT` / insert.
3. Service: map it in `mapRequestRow`, apply any rule.
4. Route: validate the new body field.
5. Client: form input + display; add a class-prefixed CSS rule.
6. Tests: a unit test for the rule and an integration case.

Add an endpoint

Route (validate + `authenticate` + `requireRole`) → service function → repository function; add an integration test using `supertest`. Keep business rules in the **service**.

Add a page

`client/src/pages/NewPage/NewPage.jsx` + `.css` (prefixed classes) → lazy import and `<Route>` in `App.jsx` wrapped in `ProtectedRoute roles={[...]}` → sidebar entry in `Sidebar.jsx` → service function in `client/src/services`.

Add a category

Insert into `public.categories` (no code change).

Change SLA targets or the overdue check interval

`server/src/utils/sla.js` (`SLA_TARGET_HOURS`) and `server/src/jobs/scheduler.js` (`OVERDUE_CHECK_EVERY_MS`). Update the tests and the user manual table.

17. Known limitations

- Integration tests fake the database (Postgres/RLS is verified in Supabase and by manual UAT); front-end browser checks are not yet an automated suite.
- Live-update connections, the profile cache and the AI rate limiter are in memory, so they assume **one server instance**.
- Free-tier hosting sleeps when idle.
- Requests cannot yet be edited or cancelled by the student after submission.
- Offline drafting covers new requests only.
- iPhone push notifications require the app to be added to the Home Screen.

18. Glossary

Term	Meaning
PWA	Progressive Web App — a website that can be installed, works offline and sends notifications
JWT	JSON Web Token — the signed proof of login sent with each API call
RLS	Row Level Security — database rules limiting which rows a user can read/write
SLA	Service-level agreement — here, the target time to resolve a request
SSE	Server-Sent Events — a one-way stream from server to browser used for live updates
VAPID	The key pair that identifies this server to browsers' push services
Service role	A Supabase key that bypasses RLS; used only by the server
Repository / Service / Route	Data-access / business-rule / HTTP layers of the backend